

Competitive Programming Notes - in C++

Maninder (Kaurman) Kaur

August 20, 2026

1 Basics of Competitive Programming

1.1 Input & Output

This is the basic format of all code:

```
#include <iostream>
#include <bits/stdc++.h>

using namespace std;

int main() {
    \\ insert content here
    return 0;
}
```

The two libraries are required, and let all libraries needed. There is no need to get other libraries, since it includes many such as `vector` or `cstring`. The line `using namespace std;` reduces redundancy. Lastly, we start our main function that always needs to return an integer value. This is where our content will lie

There are two important commands for input and output in C++. One is for input, called `cin` and the other is for output, `cout`. Always remember to add either `endl` or `\n` at the end of any output to end the value.

```
#include <iostream>
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n;
    cin >> n;
    cout << "number:" << n << endl;

    return 0;
}
```

The above code will take in a variable `n` which is an integer. The user will give this. Then we will output the text output and the number value the user gave. For example, if the user gave 4, the code will return: `$ number: 4.`

1.2 Control Statements

The most common control statement is an `if` statement. Given inputs, the function will do something if a condition is met. You can also have `if-elseif-else` or simply `if-else` statements.

```
int main() {
    int n;
    cin >> n;

    if(n > 0) {
        cout << "The value is positive" << endl;
    } else if(n == 0) {
        cout << "The value is zero" << endl;
    } else {
        cout << "The value is negative" << endl;
    }

    return 0;
}
```

We are given a number from the user. The function will now see if it is positive, zero, or negative.

1.3 Loops

Loop are a command that repeats a set of instructions until a specific condition is met. There are 3 main different types of loops: a `for` loop, a `while` loop, and a `do-while` loop.

```
int main() {
    for(int i = 1; i < 6; i++) {
        cout << i << endl;
    }
    return 0;
}
```

A `for` loop is a loop where you have a variable that will increment for a certain amount of time. In that amount of time, whatever happens inside the brackets happen. In the above example, there is a variable `i` that starts 1 and goes up till 6. The inner bracket will happen 5 times. We will then increment the variable. The output will be `$ 1 2 3 4 5.`

`for` loops can also be written as `while` and `do-while` loops as so with the previous example:

```
int main() {
    i = 1
    while (i < 6) {
        cout << i << endl;
        i++;
    }
}
```

```

    i = 1
    do {
        cout << i << endl;
        i++;
    } while (i < 6)
}

```

1.4 Functions

Functions are blocks of code that perform a specific task and can be called upon multiple times. They are useful since they can be reused over and over again. We've already seen one called `main` that return an `int` value. Let's create another and call it in `main`.

```

int adding(int n, int m) {
    return n + m;
}

int main() {
    int x, y;
    cout << "Please insert 2 numbers to add" << endl;
    cin >> x >> y;
    adding(x,y);
    return 0
}

```